1 We setup a distance - previous table:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
distance	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
previous	0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

No 0 is the current node that is lowest in distance, examine its neighboring nodes, if the current distance to the neighboring nodes is higher than the sum of the distance to the current node and the distance from the current node to its neighboring node, update it:

✓

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	∞	∞	5	∞	∞	1	∞	∞	∞	∞	∞	∞	9	∞	∞	∞
0	0	-1	-1	0	-1	-1	0	-1	-1	-1	-1	-1	-1	0	-1	-1	-1

Now lock 0 and pick 7 as it is the current node that is lowest in distance and unlocked, repeat the process:

✓ ✓

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6			5			1				10			9			
0	0			0			0				7			0			

Then:

Date: ...../...../.....

fit@hcmus

Then, pick 4:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6			5			1				10			9			
0	6			0			0				7			0			

Now, pick 1:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5			1				10			9			
0	0	1		0			0				7			0			

Now, pick 14

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5			1				10			9	10		
0	0	1		0			0				7			0	14		

Now, pick 11

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5			1	16			10	18		9	10		
0	0	1		0			0	11			7	11		0	14		

Now, pick 15.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5			1	16			10	16		9	10		
0	0	1		0			0	11			7	15		0	14		

Now, pick 2

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5	14	17	1	16			10	18		9	10		
0	0	1		0	22	0	11				7	11		0	14		

Now, pick 5, then 8 (as it doesn't change any path)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11		5	14	17	1	16	21		10	18		9	10		
0	0	1		0	2	2	0	11	5		7	11		0	14		

~~Now, pick 8.~~ Now, pick 6, 12 and then 3 and 13, then 16, 9, 10 and 17

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	6	11	19	5	14	17	1	16	21	23	10	18	19	9	10	20	26
0	0	1	6	0	2	2	0	11	5	6	7	11	12	0	14	13	13

Shortest path: (and weight):

* 0→1: 0-1 (6)
* 0→2: 0-1-2 (11)
* 0→3: 0-1-2-6-3 (19)
* 0→4: 0-4 (5)
* 0→5: 0-1-2-5 (14)
* 0→6: 0-1-2-6 (17)
* 0→7: 0-7 (1)
* 0→8: 0-7-11-8 (16)
* 0→9: 0-1-2-5-9 (21)
* 0→10: 0-1-2-6-10 (23)
* 0→11: 0-7-11 (10)
* 0→12: 0-7-11-12 (18)
* 0→13: 0-7-11-12-13 (19)
* 0→14: 0-14 (9)
* 0→15: 0-14-15 (10)
* 0→16: 0-7-11-12-13-16 (20)
* 0→17: 0-7-11-12-13-17 (26)

2. No, because Dijkstra works on a shorter path first then locks it up because it believes investigate the longer path won't lead to a shorter one, However that is no longer true if the weight is negative Example: On Node 11 is locked up after examining the path 0-7-11, However, if later we locked up node 2 and discover that 2-5 costs -3 and 5-8 costs -9, then 0-1-2-5-8-11 only costs 5 and is

the shorter path.

3 Yes, they don't assume things, they only care about weight order, so negative weight won't affect anything

4 Yes, assuming the maximum spanning tree also has minimum number of edges (V - 1), because ~~it~~ they sort the edges and pick the maximum sum in this case.